

Alberta Buck – Externally-Driven Routing Simulation

Perry Kundert

2026-05-16

1 What This Is

An *externally-driven* economic simulation: **anvil** is a hosted EVM running the **real** Alberta-Buck contracts and a **real** Uniswap V3 + Universal Router; a Python program (`alberta_buck/sim/`) owns the timeline, the actors, the price oracle, the off-chain pathfinding, and the plots. Solidity's only job is to be the BUCK system and the AMM, faithfully.

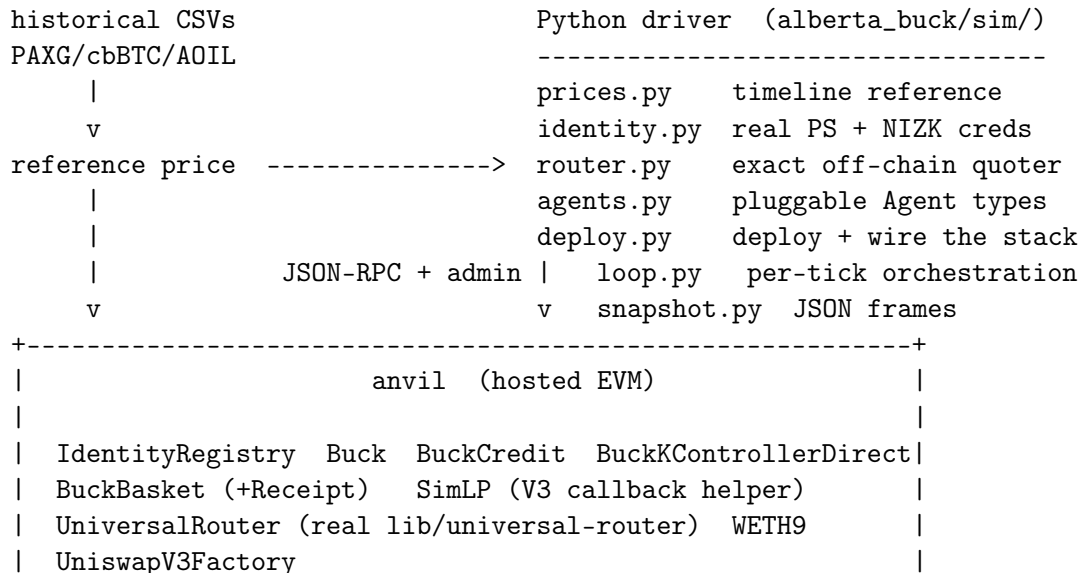
The question it answers:

Do external, BUCK-unaware, anonymous arbitrage agents – agents that only acquire and sell the underlying RWA TOKENs with USDC – naturally route through the BUCK basket pools (TOKEN -> BUCK -> TOKEN) as part of their optimized trade execution, exercising those pools as ordinary Uniswap liquidity?

A single, deep-pocketed synthetic market maker drives one TOKEN's on-chain price to its real-world (CSV) value at a random point each day; the anonymous arbitrageurs do the rest through profit-seeking optimized routing. **BUCK's own price is explicitly out of scope** – BUCK is treated as an arbitrarily-valued, unpegged token (it may sit at any value and drift freely); the focus is the routing behaviour of BUCK-unaware actors over the RWA TOKENs.

This document describes the system as built: the faithful externally-driven (`anvil` + `web3.py`) implementation in `alberta_buck/sim/`, driven by `make sim-run` / `make sim-run-rebalancing`.

2 Architecture



	PAXG/USDC	cbBTC/USDC	AOIL/USDC	(truth pools)
	PAXG/BUCK	cbBTC/BUCK	AOIL/BUCK	(basket pools)

+-----+

|

v

test/vectors/routing-sim.json -> alberta_buck/sim/plot_routing.py
-> images/routing-sim.png

Six Uniswap V3 pools. Three are the "truth" pools (TOKEN/USDC) the market maker drives to the CSV. Three are the BUCK basket pools (TOKEN/BUCK) seeded by `BuckBasket.addBasketToken + depositToken`. There is **no** BUCK/USDC pool: BUCK's absolute value is not the subject, and a BUCK-neutral router can never pin it anyway – the experiment is purely whether the BUCK pools are **used** by optimal routing. Every pool is Uniswap V3 because `BuckBasket` is V3-hardcoded.

3 The Real Contract Stack

`alberta_buck/sim/deploy.py` deploys and wires, in order (mirroring `test/BuckBasket.t.sol::setUp`):

1. `IdentityRegistry(gov); gov.trustIssuer(issuer, pspubkey).`
2. `BuckCredit().`
3. `BuckKControllerDirect(0.1e18, 0.01e18, 0, 60, 0.5e18, 1.5e18, 1e18, gov)` – the real PID, no USD intermediary.
4. `Buck(buckCredit, kCtrl, identity, insurancePool); identity.setBuck(buck).`
5. `UniswapV3Factory` (from the repo's locally-compiled v3-core).
6. `BuckBasket(buck, kCtrl, v3Factory, gov, 500, 600, 64, 50, 1000); buck.setBasket` (from the insurance pool); `kCtrl.setBasket; identity.bindContract(basket, ...).`
7. Mock ERC20s: USDC (6-dec), PAXG (18), cbBTC (8), AOIL (18).
8. `SimLP` – a ~40-line V3 mint/swap callback helper (the only new Solidity; the proven callback bodies from `UniswapV3Fixture`).
9. WETH9; the **real** `UniversalRouter` from `lib/universal-router` (staged artifact), constructed with `RouterParameters{ permit2(dummy), weth9, v2Factory=0, v3Factory, pairInitCodeHash=0, poolInitCodeHash=keccak(UniswapV3Pool bytecode), v4/migration/spoke=0 }`; `identity.bindContract`
10. Per token: create + initialize + deeply seed the TOKEN/USDC pool (via `SimLP`); `addBasketToken + depositToken` to create and deeply seed the TOKEN/BUCK pool; `bindContract` the BUCK pool.

3.1 Real cryptographic identities

Every EOA agent (every arbitrageur and the market maker) is a real, key-holding account that **self-registers** with `IdentityRegistry.register()` using an issuer-signed Pointcheval-Sanders credential and a registration NIZK bound to that EOA's own address (generated in Python by `alberta_buck/wallet/` – the same code path the Solidity verifier already accepts; see `alberta_buck/sim/identity`

Every BUCK-touching contract (`BuckBasket`, each `TOKEN/BUCK` pool, the Universal Router) is bound with a public `bindContract` Identity. After deploy the simulation asserts `IdentityRegistry.isVerified()` for every agent.

The anonymous arbitrageurs never themselves hold or move BUCK: on a `TOKEN -> BUCK -> TOKEN` leg the Universal Router is the sole BUCK counterparty and custodies the intermediate BUCK between V3 hops (the pre-fund / `payerIsUser=false` route). The agents supply and receive only USDC and RWA TOKENs, so they remain genuinely BUCK-unaware while still using the BUCK pools as part of their routes.

4 The Universal Router

The org-doc's historical worry was compiling the V4-era Universal Router. Resolved: `lib/universal-router` compiles cleanly as its own sub-project; `scripts/build_router.sh` builds it and stages `alberta_buck/sim/artifacts` which the Python driver deploys via raw bytecode. No `SimSwapper` fallback is needed – the **real** router executes every routed trade.

The router only *executes* a pre-encoded path; route *discovery* is the job of the off-chain quoter (`router.py`), exactly as in production. Our pools are full-range, so they price as constant-product; the quoter reads live pool reserves (`ERC20.balanceOf(pool)`) and computes the exact net-of-fee output of every candidate hop – the org-doc's "exact `eth_call` quoter", with no slot0 estimation error.

5 The Agents (pluggable)

`alberta_buck/sim/agents.py` defines an `Agent` base class and a name-keyed `REGISTRY`; a `Scenario` lists how many of each type to spawn. Two ship today:

- **AnonymousArbAgent** – a registered EOA holding only USDC + RWA TOKENs. Each tick it exact-quotes `USDC -> x -> BUCK -> y -> USDC` cycles over the live pools and, when one clears its margin, executes it through the Universal Router pre-fund route (BUCK router-internal; the agent never holds BUCK). Fills are tiny relative to the binding pool so slippage stays well below the detected edge – the quoter is exact, so this is non-destructive and self-limiting. This is the BUCK-unaware optimized routing that exercises the `TOKEN/BUCK` pools.
- **MarketMakerWhale** – a registered EOA with ~unlimited resources. Once per day, at a random tick, it snaps **one randomly chosen** `TOKEN/USDC` pool to that day's CSV close (never all at once), executed through `SimLP`. It is the only exogenous truth driver; the arbitrageurs propagate its move into the other pools through ordinary routing between interventions.

Adding a new agent type is mechanical: subclass `Agent`, implement `act()`, and it is auto-registered and selectable from a `Scenario` (`scenario.py`). New scenarios (an Equilibrium / Carol-Hank population, a `KArb` / Bob-Fred population, a Lifecycle) are future `Scenario` instances plus their `Agent` subclasses; `loop.py` and `snapshot.py` are scenario-agnostic.

6 Per-Tick Sequence (Python owns the clock)

per day:

```

pick a random whale-token and a random whale-tick for the day
per tick (ticks_per_day):
    anvil_setNextBlockTimestamp(t); evm_mine          # advance the clock
    if tick == whale-tick:
        whale.snap(token)    -> SimLP swap to CSV target sqrtPrice
    for each arb (randomized order -> serialized slippage):
        arb.act()             -> exact-quote cycles; best one via UR
kCtrl.compute()              # keep the PID warm
snapshot.capture(day)         # append a frame

```

Randomized per-tick agent order means an earlier agent’s fill is slippage the next agent re-quotes against – the serialized-execution realism the design calls for.

7 Running It

```

make sim-build      # stage UR artifact + emit SimLP/stack artifacts
make sim-run        # python -m alberta_buck.sim --scenario routing
                    #   (override: SIM_DAYS=365 SIM_TICKS=4)
make sim-test       # the pytest smoke wrapper (spawns its own anvil)
make sim-plot       # render images/routing-sim.png from the JSON
make sim            # sim-run + sim-plot

```

Equivalently: `python -m alberta_buck.sim --scenario routing --days 120 [--ticks-per-day K --seed S --out PATH --port P]`. Each invocation spawns its **own** anvil on an ephemeral free port with fresh state and tears it down on exit, so multiple sims run concurrently without colliding (only the default output JSON path is shared – pass `--out` to separate concurrent results).

Build note: the project’s `via_ir` setting is incompatible with recompiling the 0.7.6 v3-core trigger, so `make sim-build` scopes the Solidity build (`--skip 'src/uniswap_v3_build/*'`) and reuses the cached v3 artifacts; this is intentional and documented in the Makefile.

8 Snapshots and Plot

One JSON frame per day (`test/vectors/routing-sim.json`), schema matching `alberta_buck/sim/plot_routing` so the plot renders unchanged: per-token CSV reference, TOKEN/USDC implied price, TOKEN/BUCK implied price; `basketValueInBuck`; `buckK`; BUCK `totalSupply`; cumulative direct (whale) vs BUCK-routed trade counts; aggregate agent USDC-valued P&L.

9 Results (120-day run)

- **Real identities:** all EOA agents pass `IdentityRegistry.isVerified()` via a real issuer-signed credential
 - registration NIZK; all BUCK contracts bound.
- **BUCK pools are genuinely routed:** ~1100 BUCK-routed TOKEN -> BUCK -> TOKEN trades executed through the real Universal Router vs ~120 whale interventions – the indirect route is the arbitrageurs’ dominant path, exactly the behaviour under test.

- **TOKEN/USDC tracks the market:** with one token snapped per day and the arbitrageurs propagating via routing, TOKEN/USDC tracks the CSV reference (PAXG ~1%, cbBTC ~1-4%, AOIL looser – it is snapped on only ~1/3 of days and is the most volatile).
- BUCK’s own price drifts freely and is **not** asserted – as designed.

10 Out of Scope / Future

- BUCK price dynamics, peg, or any BUCK/USDC anchor (BUCK is treated as arbitrarily valued; not the subject).
- Additional scenarios – Equilibrium (Carol/Hank), KArb (Bob/Fred/Alice), Lifecycle – are future **Scenario** + **Agent** subclasses in the same framework; deferred.
- Multi-year horizons (the CSV supports up to its length; runs are bounded for tractability and tuned via `SIM_DAYS` vs `SIM_TICKS`).
- V4 swaps, real ZK identity **issuance** ceremony (the issuer is simulated), and on-chain route discovery (off-chain by design).